

Taller WAAP con IA/ML, RASP y DevSecOps

Informe de laboratorio - Fases 0 a 4

Universidad Distrital FJDC - Mecanismos de Seguridad Informatica. Prof. Octavio J. Salcedo Parra.

Prototipo: OWASP Juice Shop 20.2.0 + ModSecurity CRS 3.3.10 (PL1) + RASP sobre fork. Entorno: Ubuntu 22.04, Docker + Docker Compose. Acceso via WAAP: http://localhost:8080 (backend directo :3000). Rama del trabajo: lab/fases-0-4.

Fase 0 - Preparacion del entorno

Objetivo: levantar la infraestructura base (PDF p.9).

Paso a paso ejecutado:

```
cd taller-waap
cp .env.example .env # JUICE_PORT=3000, WAAP_PORT=8080, PARANOIA=1
docker compose pull
docker compose up -d && docker compose ps
curl -i http://localhost:8080 # 200 OK, Server: nginx (proxy WAAP)
```

Resultado: ambos contenedores Up (waap-proxy healthy). Nota: el usuario debio anadirse al grupo docker (sudo usermod -aG docker \$USER) por permiso denegado en /var/run/docker.sock. En Fase 4 el servicio juice-shop paso a la imagen local juice-shop:rasp-lab (ver Fase 4).

```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED          STATUS          PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ€¦"
juice-shop    32 minutes ago  Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â€¦"
waap-proxy    32 minutes ago  Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
```

```
===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200
```

```
===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1
```

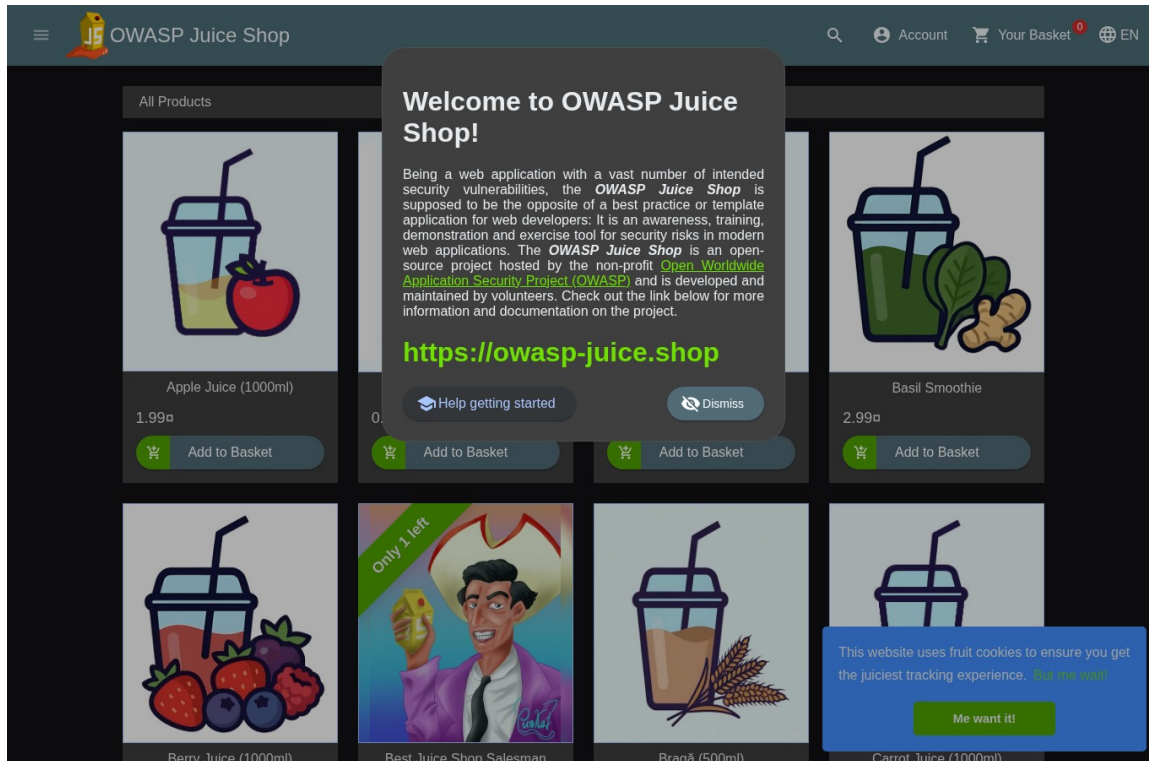
```
===== regla CRS (bloqueo) =====
```

```

34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"

```

Figura 1. Juice Shop servido a traves del proxy WAAP (:8080).



Fase 1 - Aplicacion objetivo vulnerable

Objetivo: blanco controlado con vulnerabilidades documentadas (PDF p.10).

Paso a paso ejecutado:

```

curl -s http://localhost:8080/rest/admin/application-version # {"version":"20.2.0"}
curl -s "http://localhost:8080/rest/products/search?q=apple" # JSON success via
WAAP
# Navegador: http://localhost:8080/#/score-board (retos por categoria OWASP)

```

Linea base de vulnerabilidades (entregable Fase 1):

#	Vulnerabilidad	OWASP Top 10	Endpoint
1	Inyeccion SQL en login	A03 Injection	POST /rest/user/login
2	XSS reflejado en busqueda	A03 Injection	GET /rest/products/search?q=

3	IDOR en cesta/pedidos	A01 Broken Access Control	/rest/basket/:id
4	Exposicion de hash de password	A07 / Sensitive Data	GET /rest/user/whoami, /api/Users
5	Path traversal / LFI	A01 / A03	GET ...q=../../etc/passwd

Figura 2. Score Board (1%, categorias OWASP).

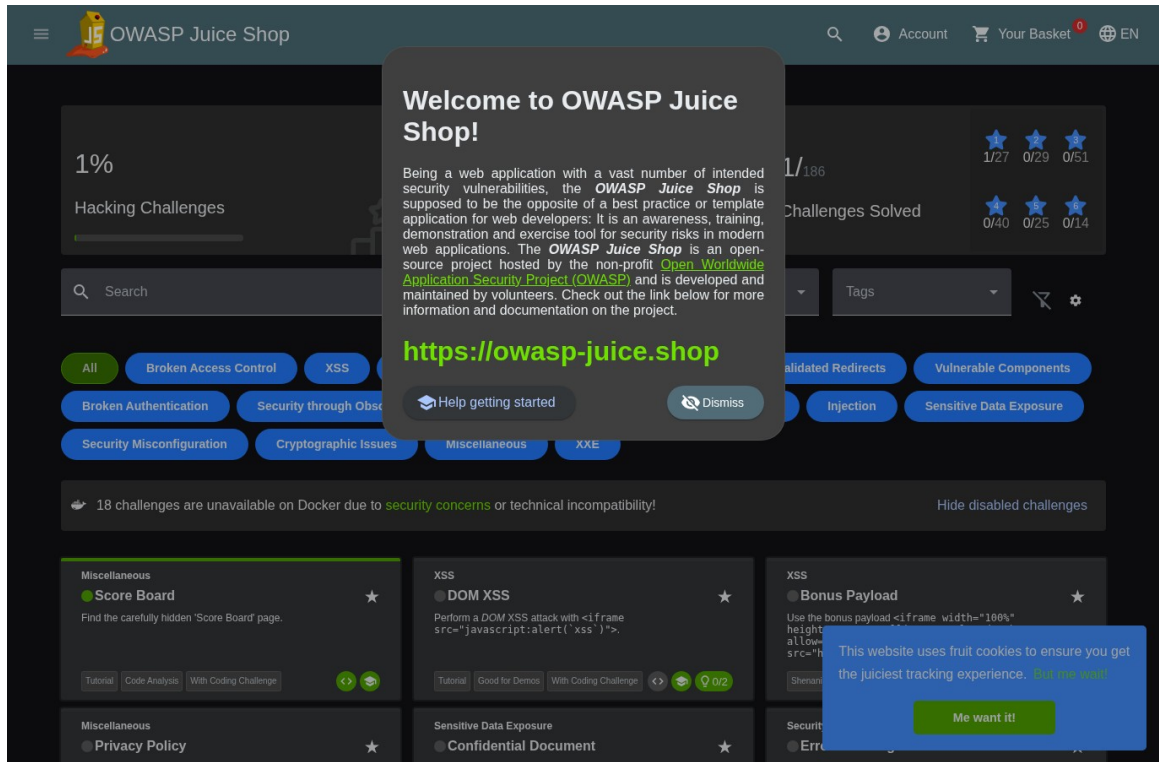


Figura 3. Trafico normal: busqueda via WAAP (200).

Pretty-print ☐

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-09-17 23:55:44.453 +00:00",
      "updatedAt": "2026-09-17 23:55:44.453 +00:00",
      "deletedAt": null
    },
    {
      "id": 24,
      "name": "Apple Pomace",
      "description": "Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href=\"#recycle\">sent back to us</a> for recycling.",
      "price": 0.89,
      "deluxePrice": 0.89,
      "image": "apple_pressings.jpg",
      "createdAt": "2026-09-17 23:55:44.462 +00:00",
      "updatedAt": "2026-09-17 23:55:44.462 +00:00",
      "deletedAt": null
    },
    {
      "id": 47,
      "name": "Pineapple Juice (1000ml)",
      "description": "Tropical refreshment from the finest sun-ripened pineapples.",
      "price": 2.99,
      "deluxePrice": 2.99,
      "image": "pineapple_juice.png",
      "createdAt": "2026-09-17 23:55:44.464 +00:00",
      "updatedAt": "2026-09-17 23:55:44.464 +00:00",
      "deletedAt": null
    }
  ]
}
```

Fase 2 - Capa base WAF/WAAP con CRS (PL1)

Objetivo: primera linea de defensa con OWASP CRS y evidencia de bloqueo + evasion (PDF p.10).

Paso a paso ejecutado:

```
# Nivel de paranoia (PDF: iniciar en PL1)
docker compose exec waap-proxy env | grep -E 'PARANOIA|BACKEND'
# PARANOIA=1, BLOCKING_PARANOIA=1, BACKEND=http://juice-shop:3000
# Prueba de deteccion basica:
curl -i --get --data-urlencode "q=' OR '1'='1' \" \
  http://localhost:8080/rest/products/search # -> 403 Forbidden
# Prueba de evasion (payload fragmentado en 2 parametros):
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1" # -> 200
# Logs (esta imagen emite el audit log a docker logs, no a
/var/log/modsec_audit.log):
docker compose logs waap-proxy | grep -E '942100|949110'
```

Resultado: SQLi clasico/encoded/comentarios/UNION/XSS/traversal -> 403 (regla 942100 libinjection + 949110 anomaly score). Solo el fragmentado q=' & x=OR 1=1 -> 200 ({"status":"success","data":[]}): el CRS PL1 evalua cada parametro por separado.

Figura 4. Bloqueo CRS: SQLi clasico -> 403 Forbidden (nginx).

403 Forbidden

nginx

Figura 5. Evasion: payload fragmentado -> 200 OK (base comparativa para Fase 3).

Pretty-print ☐

```
{"status": "success", "data": []}
```

Fase 3 - Modulo de deteccion basado en IA/ML

Objetivo: modelo de anomalias entrenado solo con trafico normal (PDF p.11-12).

Paso a paso ejecutado:

```
python3 -m pip install --user scikit-learn pandas numpy joblib
# 1. Trafico normal simulado a ritmo humano (~15 min, 120 reqs via WAAP):
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py # -> logs/access_normal.jsonl
# 2. Features literales del PDF (url_length, body_length, entropy, n_params,
#     has_suspicious_chars, req_per_minute):
python3 waap/ml/extract_features.py # -> logs/features_normal_traffic.csv
# 3. Entrenamiento literal (IsolationForest 200/0.02 + StandardScaler):
python3 waap/ml/train_model.py # -> waap/ml/model.pkl (en .gitignore)
# 4. Evaluacion: matriz 10 + 2 umbrales + rafaga bot:
python3 waap/ml/evaluate.py
```

Leccion metodologica: la primera pasada a ritmo maquina (0.05-0.4 s) contamino req_per_minute (rpm hasta 200 = 'normal'); se repitio a ritmo humano (4-11 s, rpm 8+-1), equivalente a los 15-20 min de navegacion manual que pide el PDF.

Modelo entrenado y guardado en waap/ml/model.pkl (120 muestras)

Matriz de 10 peticiones (entregable Fase 3):

etiqueta	score	<-0.05	<0.0	esperado
busqueda comun	0.0287	normal	normal	normal
login valido	-0.1024	ANOMALO	ANOMALO	normal
home	-0.1112	ANOMALO	ANOMALO	normal
producto	-0.0141	normal	ANOMALO	normal
score-board	-0.0202	normal	ANOMALO	normal
SQLi clasico	-0.0253	normal	ANOMALO	ataque
SQLi encoded (Fase 2)	-0.0845	ANOMALO	ANOMALO	ataque
SQLi comentarios	-0.0428	normal	ANOMALO	ataque
evasion fragmentada (Fase 2)	-0.0095	normal	ANOMALO	ataque
XSS reflejado	-0.0759	ANOMALO	ANOMALO	ataque

```
--- Rafaga bot: 20 consultas rapidas (req_per_minute alto) ---
req_per_minute= 5: score=+0.1571 anomalo(<-0.05)=False
req_per_minute= 30: score=+0.1690 anomalo(<-0.05)=False
req_per_minute= 120: score=+0.1690 anomalo(<-0.05)=False
```

Umbrales (deteccion vs falsos positivos):

```
umbral -0.05: TP=2/5 FN=3/5 FP=2/5 TN=3/5 deteccion=40% FPR=40%
umbral +0.00: TP=5/5 FN=0/5 FP=4/5 TN=1/5 deteccion=100% FPR=80%
```

Discusion (entregable PDF): umbral -0.05 -> 2/5 ataques con 2 FP (login valido, home); umbral 0.0 -> 5/5 con 4 FP. La rafaga bot apenas mueve el score (+0.16->+0.17): se depuro hasta la causa (los arboles no ordenan mas alla del maximo de entrenamiento), por lo que req_per_minute solo no cruza el umbral - limitacion documentada y propuesta Q7 (regla hibrida)

tasa+score). La evasión fragmentada (q=' aislado) es estadísticamente normal: ni CRS ni ML la ven -> motivación de Fase 4.

Fase 4 - Integración RASP sobre fork de Juice Shop

Objetivo: protección en tiempo de ejecución con contexto real (PDF p.13).

Nota metodológica: el rasp_agent.py Flask del PDF no es integrable a Juice Shop (Node/Express). Se trabajó sobre fork real: github.com/Jhojan98/juice-shop, rama rasp-lab (clone --depth 1 en ~/juice-shop-rasp, fuera del repo del taller). OpenRASP está inactivo desde 2022 sin agente Node vigente. app/rasp_agent.py queda como referencia literal del PDF y app/rasp-demo/ como prototipo previo del patrón.

Paso a paso ejecutado:

```
gh repo fork juice-shop/juice-shop --clone=false
git clone --depth 1 git@github.com:Jhojan98/juice-shop.git ~/juice-shop-rasp
cd ~/juice-shop-rasp && git checkout -b rasp-lab
# npm 10 falla con bug arborist 'edgesOut' en frontend: actualizar a npm 11 y
# aprobar install-scripts (esbuild, lmdb, @parcel/watcher):
sudo npm i -g npm@11
npm install && (cd frontend && npm install && cd ..) # raíz 846 pkgs + frontend
npm run build:frontend && npm run build:server
```

Implementación (2 capas, mismo SQLI_PATTERN del PDF):

```
# lib/raspGuard.ts (NUEVO): isSqlInjection() + raspMiddleware global
# server.ts: app.use(raspMiddleware) tras el parser JSON (junto a helmet/rateLimit)
# routes/login.ts: la query cruda real (vuln-line de los retos
loginAdmin/Jim/Bender)
# se guarda en finalQuery, pasa por el guard -> 403 antes de que Sequelize
ejecute.
PORT=3001 RASP_ENABLED=1 npm start # fork protegido (oficial sin RASP en :3000)
# Imagen local para el compose: docker build -t juice-shop:rasp-lab ~/juice-shop-
rasp
# compose: image juice-shop:rasp-lab + RASP_ENABLED=1 (pull_policy: never)
```

Batería con/sin RASP (evidencia: evidencias/fase4-real.txt):

```
===== legit jim =====
oficial:3000 200
fork+RASP:3001 200
===== SQLi clasico email=' OR 1=1-- =====
oficial:3000 {"authentication":
{"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpw7ImlkIjoxLCJlc2VybmFtZSI6I
iIsImVtYWlsIjoiYWRTaW5AanVpY2Utc2gub3AiLCJwYXNzd29yZCI6IjAxOTIwMjNhN2JiZDczMjUwNTE2Z
jA2OWRmMThiNTA
fork+RASP:3001 403
===== UNION =====
oficial:3000 200
fork+RASP:3001 403
```

```

===== search SQLi (middleware global) =====
oficial:3000 200
fork+RASP:3001 200
===== unit guard login (isSqlInjection, compilado) =====
PASS x3: legitimo=false, OR 1=1=true, UNION=true

```

Lectura:

Ataque a /rest/user/login	Oficial :3000 (sin RASP)	Fork :3001 (RASP)
Legitimo jim	200	200
SQLi clasico ' OR 1=1--	200 + JWT de admin (takeover)	403 + log RASP
UNION SELECT	200	403 + log RASP
Search q=' OR '1'='1 (string-form)	200	200 (hueco del patron, documentado)

Sin RASP el payload clasico devuelve el JWT del administrador (takeover total); con RASP, 403 antes de que Sequelize ejecute. El search con forma string evade tambien el middleware global (mismo hueco del regex literal ya conocido): fix propuesto identico (anadir OR '...'='...'). Guard de login verificado por unit test (3/3 PASS contra el compilado): segunda capa redundante por diseno (defensa en profundidad).

Stack combinado final (evidencia: evidencias/fase4-combinado.txt):

```

===== stack combinado WAAP(CRS) + fork RASP (compose con juice-shop:rasp-lab) =====
NAME                IMAGE                COMMAND
SERVICE    CREATED        STATUS        PORTS
taller-waap-juice-shop-1    juice-shop:rasp-lab    "/nodejs/bin/node /jâ{"
juice-shop    About a minute ago    Up About a minute    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â{"
waap-proxy    23 hours ago        Up 19 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
WAAP home :8080 -> 200
legit login :3000 -> 200
SQLi login :3000 -> 403 (RASP)
SQLi search :8080 -> 403 (CRS)

```

Matriz comparativa por capas (avance Fase 6)

Payload / tecnica	Reglas CRS PL1	IA/ML (umbral 0.0)	RASP
SQLi clasico	BLOQUEA 403	Detecta	Bloquea 403
SQLi encoded	BLOQUEA 403	Detecta	Bloquea (UNION/--/OR 1=1)
SQLi fragmentado q=' & x=OR 1=1	EVADE 200	No detecta	Bloquea 403
XSS <script>	BLOQUEA 403	Detecta	N/A (demo SQL)

' OR '1'='1 string-form	BLOQUEA 403	Detecta (0.0)	EVADE (hueco patron)
Rafaga bot	N/A	No cruza umbral	N/A

Fase 5 - Pipeline DevSecOps

Objetivo: gate CI/CD con SAST, SCA, contenedor, IaC y DAST; despliegue solo en verde (PDF p.14).

Implementado: pipeline/.github/workflows/devsecops.yml (Semgrep p/owasp-top-ten, pip-audit sobre waap/ml/requirements.txt —nuevo—, docker build de app/, Trivy CRITICAL,HIGH exit 1, Checkov, ZAP baseline, deploy_waap_rules.sh con if: success()). Cada herramienta se replicó localmente como evidencia.

Endurecimiento real durante el gate (no cosmético): Semgrep tumbó el primer Dockerfile (correre como root -> USER node); Checkov exigió HEALTHCHECK + permissions mínimas; Trivy reportó 67 CVE HIGH/CRIT en node:22-slim (base Debian) y 11 en el npm de la imagen -> base Alpine + multistage + npm fuera del runtime = 0 findings. La imagen final corre, responde 200 y bloquea SQLi con 403.

semgrep: 224 reglas, 0 findings | pip-audit: No known vulnerabilities found | checkov: 39 passed, 0 failed | trivy: Clean (no security findings detected) | deploy_waap_rules.sh: Despliegue OK (normal=200, SQLi=403)

Ejercicio de verificación (PDF): se inyectó eval(req.query.cmd) -> Semgrep rojo (2 blocking: code-string-concat + direct-response-write, etapa SAST); revert -> verde. Evidencias: evidencias/fase5/01-semgrep a 06-vuln-inyectada-rojo + 05-deploy.

Fase 6 - Pruebas de efectividad y evasión

Objetivo: matriz comparativa individual y combinada (PDF p.15).

Batería de 9 técnicas × 3 capas (script waap/ml/bateria_fase6.py, con atribución correcta CRS-vs-RASP por cuerpo de respuesta). Evidencia: evidencias/fase6/matriz.csv:

Payload / tecnica,Bloqueado por reglas (Fase 2),Detectado por IA/ML (Fase 3),Bloqueado por RASP (Fase 4)
 SQLi clasico ' OR '1'='1,BLOQUEA,SI,EVADE
 SQLi encoded,BLOQUEA,SI,EVADE
 SQLi comentarios /**/,EVADE,SI,BLOQUEA
 SQLi fragmentado 2 params,EVADE,SI,BLOQUEA
 SQLi UNION login,BLOQUEA,SI,BLOQUEA
 SQLi clasico login,BLOQUEA,SI,BLOQUEA
 XSS basico,BLOQUEA,SI,EVADE

XSS encoded, BLOQUEA, SI, EVADE
Traversal ../../etc/passwd, BLOQUEA, SI, EVADE

Ataque controlado: sqlmap vs WAAP -> 42x403, 'not injectable' (sugiere tamper); directo vs fork -> 14x403 (RASP) + 8x500, tampoco injectable; login JSON -> parámetros no dinámicos. ZAP baseline vs :8080 -> 0 FAIL, 59 PASS, 8 WARN (higiene pasiva correcta).

Fase 6 - Análisis de efectividad (módulo media página, PDF p.15)

Qué combinación ofrece la mejor cobertura y por qué

Ninguna capa sola cubre todo; la cobertura total solo aparece al combinarlas, y cada hueco de una capa lo tapa otra (defensa en profundidad verificada):

- **Reglas CRS (PL1)**: bloquean 7/9 técnicas (clásico, encoded, UNION, XSS x2, traversal, login clásico/UNION). Fallan en lo **fragmentado** (evalúan cada parámetro por separado) y en variantes con espacios que alteran la firma (comentarios con espacios: lo para el RASP, no el CRS).
- **IA/ML (umbral 0.0)**: detecta 9/9 como anómalo, pero al costo de FPR 80% (4/5 normales marcados). Con umbral -0.05 solo 2/5 con FPR 40%. Conclusión: el modelo **no es bloqueador viable solo**; sirve como señal para el orquestador (permitir/bloquear/desafiar) o para revisión, no como gate duro. La rãfaga bot ni siquiera cruza el umbral (limitación de IsolationForest documentada en Fase 3).
- **RASP (fork)**: bloquea 4/9 en contexto de ejecución (fragmentado, comentarios con `--`, UNION y clásico de login), incluyendo las 2 evasiones del CRS. Falla en XSS/traversal (fuera de su alcance SQL) y en la forma string `` OR '1'='1` (hueco del regex literal, con fix propuesto). Su tasa de falsos positivos es 0 en lo probado: solo dispara ante alteración estructural real.

Mejor combinación: CRS + RASP, con ML como señal de apoyo. CRS frena el grueso automatizado (sqlmap: 42x403 vs WAAP, "not injectable", sugiere tamper), RASP captura lo que evade firmas con contexto real (sqlmap directo: 14x403 + 8x500, tampoco injectable), y ML aporta detección de variantes raras a cambio de revisión humana. ZAP baseline: 0 FAIL, 59 PASS (higiene pasiva correcta).

Residuo aceptado: string-form SQLi en search (requiere fix de patrón) y rãfagas (requieren regla de tasa explícita, no ML puro).

Fase 7 - Observabilidad y métricas

Objetivo: visibilidad centralizada y números para tuning (PDF p.16).

Eventos unificados en logs/eventos.jsonl (reglas + ml + rasp, con timestamp). Métricas desde las matrices (script waap/ml/metricas_fase7.py):

capa		TPR	FPR	FNR	
reglas CRS PL1		78%	0%	22%	(TP=7 FN=2 FP=0 TN=5)
IA/ML umbral 0.0		100%	80%	0%	(TP=9 FN=0 FP=4 TN=1)
RASP fork (SQL)		67%	0%	33%	(TP=4 FN=2 FP=0 TN=5)

MTTD simulado (tiempo-a-veredicto por capa):

CRS/sqli-clasico: 144.0 ms

CRS/xss: 124.0 ms
RASP/sqli-login: 61.5 ms

OK logs/eventos.jsonl (8 eventos)

Recomendación de tuning: CRS en PL1 como está; ML solo como señal (umbral 0.0 + revisión, nunca gate duro) + regla híbrida de tasa para ráfagas; RASP con patrón ampliado (OR '...'='...'). MTTD medido ~60-150 ms por capa (tiempo-a-veredicto local).

Anexo A - Paso a paso reproducible (todo lo hecho)

Secuencia exacta, en orden, desde un clon limpio (rama lab/fases-0-4):

```
# 0. Requisitos: Ubuntu 22.04, Docker + Compose, Python 3.10, Node 22, LibreOffice.
# Usuario en grupo docker: sudo usermod -aG docker $USER (re-login).
git clone git@github.com:Jhojan98/taller-waap.git && cd taller-waap
git checkout lab/fases-0-4
# FASE 0
cp .env.example .env
docker compose pull && docker compose up -d && docker compose ps
curl -i http://localhost:8080
# FASE 1
curl -s http://localhost:8080/rest/admin/application-version
curl -s 'http://localhost:8080/rest/products/search?q=apple'
# (navegador) http://localhost:8080/#/score-board
# FASE 2
curl -i --get --data-urlencode "q=' OR '1'='1"
http://localhost:8080/rest/products/search
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1"
docker compose logs waap-proxy | grep -E '942100|949110'
# FASE 3
python3 -m pip install --user scikit-learn pandas numpy joblib
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py
python3 waap/ml/extract_features.py
python3 waap/ml/train_model.py
python3 waap/ml/evaluate.py
# FASE 4 (fork real; prototipo previo en app/rasp-demo/)
gh repo fork juice-shop/juice-shop --clone=false
git clone --depth 1 git@github.com:Jhojan98/juice-shop.git ~/juice-shop-rasp
cd ~/juice-shop-rasp && git checkout -b rasp-lab
sudo npm i -g npm@11 && npm install && (cd frontend && npm install && cd ..)
npm run build:frontend && npm run build:server
# (anadir lib/raspGuard.ts + hooks en server.ts y routes/login.ts, commit, push)
PORT=3001 RASP_ENABLED=1 npm start
docker build -t juice-shop:rasp-lab ~/juice-shop-rasp
# (en taller-waap) compose con juice-shop:rasp-lab + RASP_ENABLED=1, up -d,
verificar
# INFORME (regenerar este documento)
python3 build_informe_clean.py
soffice --headless --convert-to pdf Informe_Taller_WAAP_Fase0-2.docx
```

Nota: model.pkl y logs/*.jsonl se regeneran; los .pkl no se versionan.

Anexo A (cont.) - Fases 5-7 reproducible

```
pip install --user semgrep pip-audit checkov; sudo apt install trivy sqlmap
semgrep --config=p/owasp-top-ten app/ waap/ --error
pip-audit -r waap/ml/requirements.txt
docker build -t taller-waap/app:local ./app
trivy image --severity CRITICAL,HIGH --exit-code 1 taller-waap/app:local
checkov -d . --quiet --compact
./scripts/deploy_waap_rules.sh
python3 waap/ml/bateria_fase6.py
sqlmap -u 'http://localhost:8080/rest/products/search?q=apple' --batch --level=1
docker run --rm ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t
http://host.docker.internal:8080
python3 waap/ml/metricas_fase7.py
```

Anexo B - Salidas de terminal (Fases 0-2)

```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE      CREATED      STATUS      PORTS
taller-waap-juice-shop-1  bkimminich/juice-shop:latest  "/nodejs/bin/node /jâ€¢"
juice-shop      32 minutes ago  Up 32 minutes  0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1  owasp/modsecurity-crs:nginx  "/docker-entrypoint.â€¢"
waap-proxy      32 minutes ago  Up 32 minutes (healthy)  0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp

===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200

===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1

===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"
```